Ruprecht-Karls-Universität Heidelberg
Institut für Informatik

Bachelor Thesis

Parallel ASP Planning

Name: Levi Maximilian Klein
Student ID: 4283957
Supervisor: Dr. Gnad
Submission date: 17.02.2026

Ich versichere, dass ich diese Bachelor-Arbeit selbstständig verfasst und nur die angegebenen Quellen und Hilfsmittel verwendet habe.

Heidelberg, dd.mm.yyyy

Abstract

This is the abstract.

Contents

List of Figures

List of Listings

1 Introduction

Automated planning is a fundamental area of Artificial Intelligence that deals with finding a sequence of actions to transition a system from an initial state to a desired goal state. Finding an efficient and effective plan is crucial in various applications and industries such as robotics and logistics. In general, planning problems are computationally challenging, often falling into NP-hard or even PSPACE-complete complexity classes. Therefore, developing efficient algorithms and representations for planning tasks to reduce the search space and computation time is of great importance.

One of the strengths of Classical Planning is its domain independence. This means planning techniques can be used in many different fields without adjusting the core methods. This makes classical planning highly flexible and widely applicable.

1.1 Motivation

1.2 Goal of the Thesis

1.3 Structure of the Thesis

Here you describe the structure of the thesis. For example: In chapter 2, we formally define the notion of planning tasks, first describing sequential plans and afterward three different kinds of parallel plans. We then present the ASP encodings for the different plans and proofing their soundness. In chapter 3.1 we show the pipeline from translating a pddl problem to validating the given plan and introduce different optimization strategies

2 Theory

2.1 Introduction to Planning Task and Parallel Planning

Before discussing the specific encodings and optimizations for parallel planning, we establish a precise mathematical model of the planning problem.

Definition 1 ***Multi-valued planning task.*** A STRIPS-like multivalued planning task according to (Helmert 2006) is given by a 4 tuple $\langle \mathcal{F}, s_0, s_*, \mathcal{O} \rangle$, where:

- $\mathcal{F}$ is a finite set of state variables, also called **fluents**. Each $x \in \mathcal{F}$ is associated with a finite domain $\mathrm{dom}(x)$. In any given state, a fluent x is assigned exactly one value $v \in \mathrm{dom}(x)$.
- s_0, the **initial state**, is a total function,, s.t. $s_0(x) \in \mathrm{dom}(x)$ for each $x \in \mathcal{F}$, i.e. a function that maps each fluent to a value from its domain.
- s_*, the **goal conditions**, is a partial function, s.t. $s_*(x) \in \mathrm{dom}(x)$, for each $x \in \mathrm{dom}(s_*)$ i.e. a function that specifies the desired values for a subset of fluents.
- $\mathcal{O}$ is a finite set of operators, also called **actions**. An action $a \in \mathcal{O}$ is a tuple $\langle a^c, a^e \rangle$, where a^c is the *precondition* and a^e is the *postcondition* of a. Both a^c and a^e are partial functions, such that $a^c(x) \in \mathrm{dom}(x)$ for each $x \in \mathrm{dom}(a^c)$ and $a^e(x) \in \mathrm{dom}(x)$ for each $x \in \mathrm{dom}(a^e)$.

Remark. In many implementations, actions also include **conditional effects** $\mathit{eff} = \langle \mathit{cond}, x, v \rangle$, where the fluent $x \in \mathcal{F}$ is assigned value $v \in \mathrm{dom}(x)$ iff *cond*, a partial function called the *condition* such that $\mathit{cond}(x) \in \mathrm{dom}(x)$ for each $x \in \mathrm{dom}(\mathit{cond})$, holds in the current state. However, for the definitions and ASP encodings discussed in this thesis, we adopt a simplified view where effect conditions are implicitly assumed to be satisfied (i.e., mapping any effect on x to $a^e(x)$ regardless of its condition). Thus, a^e denotes the union of all variable assignments an action can potentially trigger.

Notation. Formally, the fact that a fluent X is assigned the value V in a state s is denoted by $s(X) = V$. For readability, we use the compact notation $X = V$ to denote this assignment when the reference to the specific state s is implicit from the context.

Example 1. Let us illustrate the definitions with a small example. The plan is to leave the house by opening the door, going outside, and finally closing the door.

- $\mathcal{F} = \{door, location\}$ with $\mathrm{dom}(door)$=$\{open, closed\}$, $\mathrm{dom}(location)$=$\{inside, outside\}$

- Initial state: $s_0 = \{door{=}closed, location{=}inside\}$
- Goal: $s_* = \{door{=}closed, location{=}outside\}$
- Actions: $\mathcal{O} = \{openDoor, closeDoor, goOut\}$, where

$$openDoor = \langle\{door{=}closed\}, \{door{=}open\}\rangle$$
$$closeDoor = \langle\{door{=}open\}, \{door{=}closed\}\rangle$$
$$goOut = \langle\{location{=}inside, door{=}open\}, \{location{=}outside\}\rangle$$

Intuitively, fluents can be understood as the dynamic properties that describe the configuration of the world. In our running example, this world is characterized by the *location* of the actor and the status of the *door*. The initial state captures the complete snapshot of the world at the beginning. This stands in contrast to the goal conditions, which represent the desired outcome of the planning task. Unlike the initial state, the goal is typically partial, meaning we only specify the values we explicitly care about, leaving other variables undefined. Finally, actions define the transition logic of the system: they describe how the world changes, where the precondition establishes the necessary requirements for the action to be applicable, and the postcondition specifies the resulting effects.

With these concepts in place, we now formally define the state transition function, which dictates how applying an action transforms one state into another.

Definition 2 ***Successor state.*** Let s be state and $a \in \mathcal{O}$ an action.
The *successor state* $o(a, s)$, obtained by applying action a in state s, is defined if the precondtion of a holds, i.e., $a^c(x) = s(x)$ for all $x \in \mathrm{dom}(a^c)$. Then

$$s'(x) = o(a, s) = \begin{cases} a^e(x) & \text{if } x \in \mathrm{dom}(a^e) \\ s(x) & \text{otherwise.} \end{cases}$$

Remark. Let $A_n = \langle a_1, \ldots, a_n\rangle$ be a sequence of actions with length n. We define the application of A_n to a state s recursevily by

$$o(A_n, s) = o(a_n, o(A_{n-1}, s))$$

if $o(A_i, s)$ is defined for all $1 \leq i \leq n$.

We can represent these transitions in a directed graph, where each node correspond to a distinct state, while the edges represent the possible actions from one state into another. Consequently, a solution equates to finding a path through this graph. Graphically, this transition mechanism induces a directed graph structure known as the *state space*. In this representation, the nodes correspond to the distinct states of the world, while the edges represent the actions transforming one state into another. Consequently, finding a solution equates to identifying a path through this graph. Since this graph grows exponentially with the number

of actions, efficiently searching through this state space is a central challenge in automated planning.

Definition 3 ***Sequential plan.*** A *sequential plan* is a sequence of actions $A_n = \langle a_1, \ldots, a_n \rangle$, s.t. $s' = o(A_n, s_0)$ is defined and $s'(x) = s_*(x)$ for all $x \in \mathrm{dom}(s_*)$.

Example 2. Consider the planning task from the previous example. One possible sequential plan is $A = \langle openDoor, goOut, closeDoor \rangle$. Since

$$\begin{aligned} o(A, s_0) &= o(\langle openDoor, goOut, closeDoor \rangle, \{door{=}closed, location{=}inside\}))) \\ &= o(\langle goOut, closeDoor \rangle, \{door{=}open, location{=}inside\})) \\ &= o(closeDoor, \{door{=}open, location{=}outside\}) \\ &= \{door{=}closed, location{=}outside\} = s_* \end{aligned}$$

While the sequential approach is intuitive, it is often inefficient searching through the whole state space. A possible way to reduce the horizon required to find a solution is to allow multiple actions to be executed simultaneously, forming a parallel plan. However, arbitrary sets of actions cannot simply be executed simultaneously, since conflicts may arise if actions interfere with each other. The most fundamental requirement for a set of actions to be valid is that they must agree on their postcondition. If two actions executing in parallel attempt to assign different values to the same variable, the resulting state would be undefined. To prevent this ambiguity, we introduce the concept of *confluence*.

Definition 4 ***confluent.*** Let $A = \{a_1, \ldots, a_n\} \subseteq \mathcal{O}$ be a set of actions.
A is *confluent* $:\Longleftrightarrow a_i^e(x) = a_j^e(x)$ for all $1 \leq i < j \leq n$ and each $x \in \mathrm{dom}(a_i^e) \cap \mathrm{dom}(a_j^e)$

Remark. A is confluent $\Leftrightarrow$ B is confluent for all $B \subseteq A$

Example 3. In our initial example, the set $A = \{openDoor, goOut\}$ is confluent because the domains of their effects are disjoint. Similarly, $\{closeDoor, goOut\}$ is confluent.

To illustrate more complex scenarios, we extend our example with two fluents, *light* and *key*, where $\mathrm{dom}(light) = \mathrm{dom}(key) = \{0, 1\}$, and two corresponding actions:

$$\begin{aligned} lightsOut &= \langle \{location{=}inside, light{=}1\}, \{light{=}0\} \rangle \\ getKey &= \langle \{location{=}inside, key{=}0\}, \{key{=}1\} \rangle \end{aligned}$$

Additionally, we modify the precondition of *openDoor* to require the key ($openDoor^c = \{door{=}closed, key{=}1\}$) and update the goal to $s_* = \{location{=}outside, door{=}closed, light{=}0\}$.

In this extended setting, the set $A' = \{getKey, openDoor, goOut, lightsOut\}$ is also a confluent set of actions. Even though the actions are semantically related, their effects target different variables ($key, door, location, light$), so no conflicts in their effects occur.

The concept of confluence ensures that the simultaneous execution of actions within a set A does not lead to contradictory state updates regarding the postcondition of the actions $a \in A$. However, ensuring consistent effects is only one part of the problem, since we also need to consider the preconditions of the actions. Unlike in sequential planning, where preconditions are simply checked against the current state, parallel planning allows for different interpretations of precondition dependencies. These interpretations lead to the different parallel representations of sequential plans, which have been investigated in the literature (Dimopoulos, Nebel, and Koehler 1997); (Rintanen, Heljanko, and Niemelä 2006); (Wehrle and Rintanen 2007) and are specifically implemented and tested using Answer Set Programming in (Dimopoulos, Gebser, et al. 2019).

Definition 5 ***Parallel representations.*** Let s be a state and $A = \{a_1, \ldots, a_n\}$ be a confluent set of actions. Then A is called:

(i) $\forall$-*step serializable* in s $:\Longleftrightarrow$ $o(\langle a_{\pi(1)}, \ldots, a_{\pi(n)} \rangle, s)$ is defined for **all** permutations π

(ii) $\exists$-*step serializable* in s $:\Longleftrightarrow$ $o(\langle a_{\pi(1)}, \ldots, a_{\pi(n)} \rangle, s)$ is defined for **some** permutation π and $a^c(x) = s(x)$ for each $a \in A$, $x \in \mathrm{dom}(a^c)$

(iii) *relaxed* $\exists$-*step serializable* in s $:\Longleftrightarrow$ $o(\langle a_{\pi(1)}, \ldots, a_{\pi(n)} \rangle, s)$ is defined for **some** permutation π

Remark. A is $\forall$-step serializable $\Rightarrow$ A is $\exists$-step serializable $\Rightarrow$ A is relaxed $\exists$-step serializable.

Each parallel representation has different restrictions on how the preconditions of the actions in A are satisfied with $\forall$-step being the most restrictive and relaxed $\exists$-step being the least restrictive.

- **$\forall$-step serialization** requires strong independence: Since the set must be executable in *any* order, no action is allowed to invalidate the preconditions of another action in the same set and since any action $a \in A$ must be executable at the start, all preconditions have to be satisfied in the current state s.
- **$\exists$-step serialization** relaxes the independence. Preconditions must still be satisfied in s (i.e., actions cannot enable each other). However, conflicts are allowed: One action $a \in A$ may invalidate the precondition of another action $a' \in A$, provided that there exists *some* valid execution order where a comes after a'.
- **Relaxed $\exists$-step serialization** is the least restrictive. It drops the requirement that preconditions must hold in s. An action $a \in A$ can establish the preconditions required by another action $a' \in A$, as long as they can be ordered such that the a comes before a'.

Based on these definitions of valid parallel steps, we can now define the corresponding parallel

plans.

Definition 6 ***Parallel Plans.*** Let $A = \langle A_1, \ldots, A_m \rangle$ be a sequence of confluent sets of actions. A is a *∀-step, ∃-step or relaxed ∃-step plan* iff

- $s_m(x) = s_*(x)$ for each $x \in \mathrm{dom}(s_*)$
- A_i is ∀-step, ∃-step or relaxed ∃-step serializable in s_{i-1} for $1 \leq i \leq m$, where

$$s_i(x) = \begin{cases} a^e(x) & \text{if } x \in \mathrm{dom}(a^e) \\ s_{i-1}(x) & \text{otherwise} \end{cases}$$

  for each $a \in A_i$.

Example 4. In our modified example we have:

- ∀-step plan: $\langle \{getKey, lightsOut\}, \{openDoor\}, \{goOut\}, \{closeDoor\} \rangle$
- ∃-step plan: $\langle \{getKey, lightsOut\}, \{openDoor\}, \{goOut, closeDoor\} \rangle$
- relaxed ∃-step plan: $\langle \{getKey, lightsOut, openDoor, goOut\}, \{closeDoor\} \rangle$

While the concept of a sequential plan is straightforward, we are effectively dealing with transitions of the form $s \xrightarrow{a} s'$. In parallel planning, however, we generalize this transition to allow sets of actions, i.e., $s \xrightarrow{A} s'$, effectively merging multiple actions together into a single one.

Summary

In this section, we have established the formal framework for multi-valued planning tasks. We defined the static components of the world (fluents and states), the dynamic transition logic (applying actions), and the conditions under which sets of actions can be executed in parallel. With these formal definitions in place, the next chapter will demonstrate how to translate this theoretical framework into concrete Answer Set Programming encodings to solve planning tasks efficiently.

2.2 Introduction to ASP

Answer Set Programming (ASP) is a declarative problem-solving approach oriented towards difficult (primarily NP-hard) search problems. Unlike imperative programming, where the programmer specifies the control flow to compute a solution, ASP follows a declarative paradigm: the problem is described as a logic program such that its valid models—called *answer sets*—correspond to the solutions of the original problem. This methodology allows for a clean separation between the problem description (the logic program) and the solving algorithms (implemented by ASP systems like *clingo*).

The theoretical foundation of ASP lies in the stable model semantics of logic programming (Gelfond and Lifschitz 1988). To formally define these programs, we first introduce their basic

building blocks. The most fundamental units are *atoms* of the form $p(t_1, \ldots, t_n)$ or p, which can either be true or false. These atoms are combined to form literals. A literal is defined as either an atom A (a positive literal) or its default negation *not* A (a negative literal). Here, *not* denotes negation as failure, meaning that *not* A is considered true if no proof for A can be derived from the program.

Logic Programs

The head of a rule or fact is an atom, which has the same syntatic form as a constant or a function, i.e. $prec(X, V, t)$. The body consists of a (possible empty) list of literals $L_1, \ldots L_n$. Each L_i is of the form A or *not* A, where A is an atom and *not* the default negation. The ":-" operator can be read as "⇐", therefore a rule can be interpreted as "if all literals in the body are true, then the head is true". A fact is a rule without a body, meaning that the head atom is unconditionally true. An integrity constraint is a rule without a head, that filters solution candidates, meaning that all the literal in its body must not be satisfied, i.e. at least one literal in the body must be false. A set of atoms is called a *model* of a logic program, if all facts, rules and integrity constraints are satisfied. Atoms are considered true if they are in the model, otherwise they are considered false. In ASP, a model is called an *answerset* or *stablemodel*, if every atom in the model has an acyclic derivation of the program.

Based on these definitions, we can specify the structure of the programs used to model planning tasks. A *normal logic program* consists of a finite set of rules, facts, and integrity constraints. These constructs generally follow the form:

$$\begin{aligned} \textbf{Fact:} \quad & A_0. \\ \textbf{Rule:} \quad & A_0 \text{ :- } L_1, \ldots, L_n. \\ \textbf{Constraint:} \quad & \text{ :- } L_1, \ldots, L_n. \end{aligned}$$

The *head* of a rule is an atom A_0. The *body* consists of a (possibly empty) conjunction of literals $L_1, \ldots, L_n$. The symbol " :- " stands for logical implication ($\leftarrow$) and allows the rule to be interpreted as: "if all literals in the body are true, then the head must be true".

We distinguish three types of rules:

- A **fact** is a rule with an empty body, implying that the head atom A_0 is unconditionally true.
- A **rule** derives the head A_0 if the body is satisfied.
- An **integrity constraint** is a rule with an empty head. It serves as a filter for solution candidates: At least one literal in the body must be false,otherwise the candidate model is rejected.

A set of atoms is called a model of a logic program if it satisfies all rules, facts, and integrity constraints. Atoms are considered true iff they are contained in the model. In ASP, a model is

called an answer set if every atom in the model has an acyclic derivation from the program. Based on these semantics, we can now translate the formal definition of a planning task into an ASP logic program. The problem instance is represented purely by facts: the initial state s_0 is defined via `init/2`, the goal s_* via `goal/2`, and the pre- and postconditions of actions via `prec/3` and `post/3` respectively. Listing 1 shows the complete fact representation for our running example.

```
1  fluent(door; loc; light; key).
2  value(door, 0).  value(door, 1).
3  value(loc, in).  value(loc, out).
4  value(light, 0). value(light, 1).
5  value(key, 0).   value(key, 1).
6
7  init(door, 0). init(loc, in).  init(light, 1). init(key, 0).
8  goal(door, 0). goal(loc, out). goal(light, 0).
9
10 action(openDoor; closeDoor; goOut; lightOut; getKey).
11 prec(openDoor, door, 0).  prec(openDoor, key, 1).   post(openDoor, door, 1).
12 prec(closeDoor, door, 1).                           post(closeDoor, door, 0).
13 prec(goOut, loc, in).     prec(goOut, door, 1).  post(goOut, loc, out).
14 prec(lightOut, loc, in).  prec(lightOut, light, 1). post(lightOut, light, 0).
15 prec(getKey, loc, in).    prec(getKey, key, 0).     post(getKey, key, 1).
```

Listing 1: ASP fact representation of the example planning task

Combining these facts with the right encodings, stable models correspond to the *sequential,* $\forall$*-step,* $\exists$*-step* or *relaxed* $\exists$*-step plans.*

```
1 #include <incmode>.
2 holds(X, V, 0) :- init(X, V).
3 #program check(t).
4 :- query(t), goal(X, V), not holds(X, V, t).
5 #program step(t).
6 {holds(X, V, t) : value(X, V)} = 1 :- fluent(X).
7 {occurs(A, t)} :- action(A).
8 :- occurs(A, t), post(A, X, V), not holds(X, V, t).
9 :- holds(X, V, t), not holds(X, V, t-1), not occurs(A, t) : post(A, X, V).
```

Listing 2: Core for sequential and parallel encodings for planning

All planning algorithms presented in this work share a common incremental encoding using the *incmode* feature of *clingo* (Gebser, Kaminski, Kaufmann, and Schaub 2014). The program is divided into three subprograms: *base*, *step(t)* and *check(t)*.

The ***base*** subprogram (Line 2) defines the initial state s_0, grounding the fluent values at time step 0. The predicate $holds(X, V, t)$ is used to represent fluents $X \in \mathcal{F}$ that have value $V \in \text{dom}(X)$ at timestep t.

The ***check(t)*** subprogram (Line 4) is a parametrized subprogram starting at $t = 0$. The integrity constraint checks if the goald condition is met, i.e. $s_*(X) = s_t(X)$ for each $X \in \text{dom}(s_*)$. The external atom $query(t)$ ensures this check is only active for the current search horizon t.

The ***step(t)*** subprogram (Line 6-9) is grounded for each timestep t starting at $t = 1$. Note that postconditions of applied actions in timestep t become effective in t, while their preconditions have to hold in $t - 1$. Line 6 generates the state candidates for timestep t, assigning every fluent X exactly one value V from its domain. Line 7 generates arbitrary sets of actions $occurs(A, t)$ for the current timestep. These actions and the generated state must obey the constraints in Lines 8 and 9. Line 8 ensures that the postcondition of an occurring action is actually applied in the state. This also asserts the confluence of the set of actions: if the postconditions of two actions have different values for the same fluent ($V = a^e(X) \neq a'^e(X) = V'$), the uniqueness of the value in the choice rule (Line 6) would be violated. Finally, Line 9 guarantees that fluents not affected by any action keep their value from the previous timestep. It rejects any change in a fluent's value that is not explained by an action.

In summary, Listing 2 establishes the state transitions consistent with the postcondition of the chosen actions. However, it does not yet verify the *applicability* of these actions regarding their preconditions, nor does it enforce the specific serializability requirements (e.g., acyclicity) of the parallel semantics.

2.2.1 Extension for sequential plans

To restrict the generic transition system to valid *sequential plans*, we need to ensure that preconditions are satisfied and that each execution is restricted to a single action at a time. This is achieved by the extension shown in Listing 3.

```
1 :- occurs(A, t), prec(A, X, V), not holds(X, V, t-1).
2 :- #count{A : occurs(A, t)} > 1.
```

Listing 3: Extension of Listing 2 for encoding sequential plans

Line 2 implements the applicability check: it enforces that for any action occurring at timestep t, all its preconditions must hold in the preceding state s_{t-1}. Line 3 ensures that only one action can occur in each timestep t.

Example 5. Running this with our example, we get three possible sequential plans.

```
Answer: 1
occurs(getKey,1) occurs(lightOut,2) occurs(openDoor,3) occurs(goOut,4) occurs(closeDoor,5)
Answer: 2
occurs(getKey,1) occurs(openDoor,2) occurs(lightOut,3) occurs(goOut,4) occurs(closeDoor,5)
Answer: 3
occurs(lightOut,1) occurs(getKey,2) occurs(openDoor,3) occurs(goOut,4) occurs(closeDoor,5)
```

Listing 4: Output from clingo for sequential encoding

2.2.2 Extension for ∀-step plans

```
1 :- occurs(A, t), prec(A, X, V), not holds(X, V, t-1).
2 :- occurs(A, t), prec(A, X, V), not post(A, X, _), not holds(X, V, t).
3 single(X, t) :- occurs(A, t), prec(A, X, V1), post(A, X, V2), V1 != V2.
4 :- single(X, t), #count{A : occurs(A, t), post(A, X, V)} > 1.
```

Listing 5: Extension of Listing 2 for encoding ∀-step plans

The extension for ∀-step plans is shown in Listing 5. Line 1 is identical to the sequential encoding, ensuring that the preconditions of all chosen actions are satisfied in the previous state

s_{t-1}. To create a valid ∀-step plan, we must ensure that each set of actions is executable in *any* order. Since Line 8 of the common core (Listing 2) already enforces confluence, the remaining requirement is that no action is allowed to invalidate the preconditions of another action in the same set. We distinguish two types of interactions between an action a and a fluent X in its precondition ($\mathrm{dom}(a^c)$):

1. The action has X in its preconditions fluent X in its precondition but does not change it, (i.e. $X \in \mathrm{dom}(a^c) \cap \mathrm{dom}(a^e)$ and $a^c(X) = a^e(X)$ or $X \in \mathrm{dom}(a^c)$ and $X \notin \mathrm{dom}(a^e)$).
2. The action has fluent X in its preconditions and changes its value (i.e., $X \in \mathrm{dom}(a^e)$ and $a^c(X) \neq a^e(X)$).

Line 2 handles case 1 by ensuring that X has to remained unchanged. However, case 2 requires strict isolation of these kinds of actions. If an action a invalidates its own precondition, no other action a' in the same timestep is allowed to modify X. If such an action a' existed, the execution order $\langle a', a\rangle$ would fail because a' would change X, invalidating the precondition of a. To enforce this, Line 3 marks such actions using the predicate `single`(X, t). The integrity constraint in Line 4 then ensures that for any fluent X, at most one such action occurs per time step, preventing any interference.

Example 6. Running this with our example, we get two possible ∀-step plans.

```
Answer: 1
occurs(getKey,1) occurs(openDoor,2) occurs(lightOut,2) occurs(goOut,3) occurs(closeDoor,4)
Answer: 2
occurs(lightOut,1) occurs(getKey,1) occurs(openDoor,2) occurs(goOut,3) occurs(closeDoor,4)
```

Listing 6: Output from clingo for ∀-step encoding

2.2.3 Extension for ∃-step plans

```
1 :- occurs(A, t) , prec(A, X, V), not holds(X, V ,t -1).
2 apply(A1, t) :- action(A1),
3     ready(A2, t) : post(A1, X, V1), prec(A2, X, V2), A1 != A2, V1 != V2.
4 ready(A, t) :- action(A), not occurs(A, t).
5 ready(A, t) :- apply(A, t).
6 :- action(A) , not ready(A, t).
```

Listing 7: Extension of Listing 2 for encoding ∃-step plans

```
1 :- occurs(A, t), prec(A ,X ,V), not holds(X, V, t-1).
2 #edge((A1, t), (A2, t)): occurs(A1, t),
3     post(A1, X, V1), prec(A2, X, V2), A1 != A2, V1 != V2.
```

Listing 8: Alternative Extension for ∃-step plans

There are two different ways to encode the extension for ∃-step plans as seen in Listing 7 and Listing 8. As before, Line 1 in Listing 7 and 8 is the same as in the sequential and ∀-step encoding. To ensure that there is at least one valid permutation, either an action a_1 that invalidates the precondition of another action a_2 has to be applied after a_2 or action a_2 does not occur in this timestep. For this, all actions has to be marked as *ready* (Line 6). An action is *ready* if it either does not occur in this timestep (Line 4) or if it is captured by *apply* (Line 2). An action can be safely applied, if all other actions whose preconditions it invalidates are ready, i.e. are either applied beforehand or not occurring in this timestep. Finally, Line 6 ensures that actions does not circularly invalidates each other.

Dimopoulos, Gebser, et al. (2019) presents two different approaches to encode the extension for ∃-step plans, shown in Listings 7 and 8. As before, Line 1 in both listings enforces the standard precondition check. The difference lies in how they enforce the serializability requirement: the set of parallel actions must allow for at least one valid permutation where no action invalidates the precondition of a subsequent action.

Listing 8 implements a graph-based approach. Since the common core (Listing 2) already establishes confluence, the only remaining requirement is the existence of a valid execution order. This effectively means excluding circular interference among the chosen actions.

This lends itself to an implementation using the `#edge` directive (Gebser, Kaminski, Kaufmann, Ostrowski, et al. 2016) of *clingo*, where built-in acyclicity checking (Bomanson et al. 2016) is utilized.

The underlying structure corresponds to the *conflict graph*. Following the definition in Dimopoulos, Gebser, et al. (2019), we define the graph over all possible actions, with edges originating from the currently active set.

Definition 7 ***Conflict Graph.*** Let A_t be the set of actions occurring at time t. The conflict graph $G_t = (\mathcal{O}, E)$ for timestep t is a directed graph where

$$E = \{\langle a, a' \rangle \mid \{a, a'\} \in A_t \times \mathcal{O} \setminus \{a\}, \exists X \in \mathrm{dom}(a^e) \cap \mathrm{dom}(a'^c) : a^e(X) \neq a'^c(X)\}$$

Intuitively, an edge $(a, a') \in E$ exists if action a invalidates a precondition of action a'. Line 2 of Listing 8 directly translates these edges into the `#edge` syntax. A valid serialization of the actions corresponds to a reversed topological ordering of the subgraph induced by

A_t. Consequently, any answer set containing a cycle within the set of occurring actions is automatically rejected by the solver's cycle-checking logic.

Listing 7, on the other hand, implements this acyclicity checking by a recursive derivation of the predicates `apply` and `ready`. Line 4 defines the base case: Any action that does not occur is considered *ready* (since it does not interfere with the execution order). Line 5 marks applied actions: If an action is successfully applied, it is also `ready`. Line 2-3 implements the ordering logic. An action a_1 can only be applied (`apply`) if all other actions a_2 whose preconditions it invalidates are already applied or do not occur (`ready`). Line 6 is the final constraint: It requires that every occurring action must eventually become *ready*. If the subgraph induced by A_t contains a cycle (e.g., a_1 invalidates a_2 and a_2 invalidates a_1), neither action can ever be derived as `apply`, because each would wait for the other to be ready first. Consequently, the constraint in Line 6 would be violated, causing the solver to reject the model.

Example 7. Running this with our example, we get three possible ∃-step plans.

```
Answer: 1
occurs(getKey,1) occurs(openDoor,2) occurs(closeDoor,3) occurs(goOut,3) occurs(lightOut,3)
Answer: 2
occurs(getKey,1) occurs(openDoor,2) occurs(lightOut,2) occurs(closeDoor,3) occurs(goOut,3)
Answer: 3
occurs(lightOut,1) occurs(getKey,1) occurs(openDoor,2) occurs(closeDoor,3) occurs(goOut,3)
```

Listing 9: Output from clingo for ∃-step encoding

2.2.4 Extension for relaxed ∃-step plans

```
1 reach(X, V, t) :- holds (X, V, t -1).
2 reach(X, V, t) :- occurs(A, t), apply(A, t), post(A, X, V).
3 apply(A1, t) :- action(A1), reach(X, V, t) : prec(A1, X, V);
4     ready(A2, t) : post(A1, X, V1), prec(A2, X, V2), A1 != A2 , V1 != V2.
5 ready(A, t) :- action(A), not occurs(A, t).
6 ready(A, t) :- apply(A, t).
7 :- action(A), not ready(A, t).
```

Listing 10: Extension of Listing 2 for encoding relaxed ∃-step plans

Finally, we present the extension from Dimopoulos, Gebser, et al. 2019 for relaxed ∃-step plans in Listing 10. Structurally, it shares the same derivation approach with the standard

∃-step encoding, but it relaxes the applicability check. Previously, preconditions strictly had to hold in the previous state s_{t-1}. Here, actions within the same parallel set can provide "internal support" for each other. This means an action is applicable if its preconditions are established either by the previous state or by another action occurring earlier in the valid execution sequence of the current step.

This logic is implemented as follows: The predicate `reach`(X, V, t) is introduced to represent the values of fluents that are currently available. A value is reachable if it holds in the previous state (Line 1) or if it is obtained by an action that has already been applied in the current internal sequence (Line 2). The `apply` rule now checks against this `reach` predicate instead of the static state. An action can be safely applied if its preconditions are reachable. As in the standard ∃-step encoding, we must still check acyclicity. The integrity constraint in Line 7 ensures that no circular dependencies exist, guaranteeing a valid serialization exists.

Example 8. Running this with our example, we get three possible *relaxed* ∃-step plans.

```
Answer: 1
occurs(openDoor,1) occurs(getKey,1) occurs(closeDoor,2) occurs(goOut,2) occurs(lightOut,2)
Answer: 2
occurs(openDoor,1) occurs(lightOut,1) occurs(getKey,1) occurs(closeDoor,2) occurs(goOut,2)
Answer: 3
occurs(openDoor,1) occurs(goOut,1) occurs(lightOut,1) occurs(getKey,1) occurs(closeDoor,2)
```

Listing 11: Output from clingo for relaxed ∃-step encoding

2.3 Proof of Correctness

In order to prove the correctness of the presented encondings, Dimopoulos, Gebser, et al. 2019 defines the following:

- B is the rule in Line 1 of Listing 2.
- $Q(t)$ is the integrity constraint in the check program directive (Line 5) of Listing 2.
- $S(t)$ are the rules and integrity constraint in the step program directive (Lines 6-9) of Listing 2.
- $S^p(t)$ is the extension for the specific plans, where $p = s$ for Listing 3, $p = \forall$ for Listing 5, $p = \exists$ for Listing 7, $p = E$ for Listing 8 and $p = R$ for Listing 10.
- I is the set of facts representing a planning task $\langle \mathcal{F}, s_0, s_*, \mathcal{O} \rangle$.
- $\langle a_1, \ldots, a_n \rangle$ is a sequence of actions.
- $\langle A_1, \ldots, A_m \rangle$ is a a sequence of sets of actions.
- $P^p(n) := I \cup B \cup Q(0) \cup \bigcup_{t=1}^{n} (Q(t) \cup S(t) \cup S^p(t)) \cup \{\texttt{query}(n).\}$, where $p \in \{s, \forall, \exists, E, R\}$.

Having presented the ASP encodings for sequential and the different parallel semantics, we now proceed to formally verify their correctness. The goal is to establish a one-to-one correspondence between the stable models of our logic programs and the valid plans as defined in Definition 6. We adopt the notation for the proof from Dimopoulos, Gebser, et al. (2019) and decompose the logic programs into their constituent parts.

- I denotes the set of facts representing the specific planning task instance $\langle \mathcal{F}, s_0, s_*, \mathcal{O} \rangle$ (as in Definition 1).
- B corresponds to the rules in the `base` subprogram (Listing 2, Line 2).
- $Q(t)$ represents the goal verification rules defined in the `check(t)` subprogram (Listing 2, Line 4).
- $S(t)$ denotes the transition rules (generator and core constraints) defined in the `step(t)` subprogram (Listing 2, Lines 6–9).
- $S^p(t)$ is the specific extension for timestep t, where:
  - $p = \text{s}$ for the sequential extension (Listing 3),
  - $p = \forall$ for the $\forall$-step extension (Listing 5),
  - $p = \exists$ for the constructive $\exists$-step extension (Listing 7),
  - $p = \text{E}$ for the graph-based $\exists$-step extension (Listing 8),
  - $p = \text{R}$ for the relaxed $\exists$-step extension (Listing 10).

Using these components, the complete logic program for a horizon n is defined as:

$$P^p(n) := I \cup B \cup \bigcup_{t=1}^{n} (S(t) \cup S^p(t) \cup Q(t)) \cup \{\texttt{query}(n).\}$$

Theorem 1 (Correctness of sequential plans (Dimopoulos, Gebser, et al. 2019))**.** Let $A_n := \langle a_1, \ldots, a_n \rangle$, $a_t \in \mathcal{O}$ for $1 \leq t \leq n$, be a sequence of actions. Then,

$$A_n \text{ is a sequential plan} \iff P^s(n) \text{ has a stable model } M \text{ such that } \{\langle a, t \rangle \mid \texttt{occurs}(a, t) \in M\} = \{\langle a_t, t \rangle \mid 1 \leq t \leq n\}.$$

Proof. We prove both directions separately.

($\Rightarrow$): Assume $A_n = \langle a_1, \ldots, a_n \rangle$ is a sequential plan.
We construct a candidate set M based on the plan execution and show it is indeed a stable model of $P^s(n)$. The rule in B ensures that `holds`(X,V,0) $= s_0(X)$ for each $X \in \mathcal{F}$. The choice rule (Line 5 of Listing 3) enforces that for any $0 \leq t \leq n$ and $X \in \mathcal{F}$, exactly one value V is assigned, i.e., $|\{V \mid \texttt{holds}(X, V, t) \in M\}| = 1$. Moreover, the integrity constraints (Lines 8 and 9) in $S(t)$ dictate the transition logic between states. For any

$1 \leq t \leq n$, the value V of a fluent X is determined by:

$$\texttt{holds}(X, V, t) \leftrightarrow \begin{cases} V = a_t^e(X) & \text{if } X \in \text{dom}(a_t^e) \\ V = s_{t-1}(X) & \text{otherwise} \end{cases}$$

Therefore, the set of `holds`-atoms in any stable model must satisfy:

$$\{\langle X, V, t\rangle \mid \texttt{holds}(X, V, t) \in M\} = \{\langle X, V, t\rangle \mid X \in \mathcal{F}, V{=}o(A_n, s_0)(X), 0 \leq t \leq n\} \quad (1)$$

Since the action sequence is fixed by the plan assumption ($\{\langle a, t\rangle \mid \texttt{occurs}(a, t) \in M\} = \{\langle a_t, t\rangle \mid 1 \leq t \leq n\}$), the only candidate for a stable model $P^s(n)$ is:

$$M = I \cup \{\texttt{occurs}(a_t, t) \mid 1 \leq t \leq n\} \\ \cup \{\texttt{holds}(X, V, t) \mid (X, V, t) \text{ satisfies } (1)\} \cup \{\texttt{query}(n)\}.$$

By the definition of a sequential plan, $o(a_t, s_{t-1})$ is defined for all $1 \leq t \leq n$, and the goal condition $s_n(X) = s_*$ for each $X \in \text{dom}(s_*)$ is met. Thus, the integrity constraint regarding preconditions (Line 1, Listing 3) and the goal check (Line 4, Listing 2) are satisfied. Since M is a set satisfying all rules and constraints, it is indeed a stable model of $P^s(n)$.

($\Leftarrow$): Assume that M is a stable model of $P^s(n)$ s.t. $\{\langle a, t\rangle \mid \texttt{occurs(a,t)} \in M\} = \{\langle a_t, t\rangle \mid 1 \leq t \leq n\}$.
We prove by induction over t, that for each $1 \leq t \leq n$, the following holds:

$$\{\langle X, V\rangle \mid \texttt{holds(X,V,t)} \in M\} = \{\langle X, V\rangle \mid X \in \mathcal{F}, V = o(\langle a_1, \ldots, a_t\rangle, s_0)(X)\} \quad (2)$$

Base case $t = 0$:
The rule in B directly implies that $\{\langle X, V\rangle \mid \texttt{holds(X, V, 0)} \in M\} = \{\langle X, V\rangle \mid X \in \mathcal{F}, s_0(X) = V\}$, satisfying the equation for $t = 0$.
Induction step $t - 1 \mapsto t$:
Assume the hypothesis holds for $t - 1$, i.e., the state at $t - 1$ corresponds to $s_{t-1} = o(\langle a_1, \ldots, a_{t-1}\rangle, s_0)$.. The integrity constraint in Line 1 of Listing 3 in $S^s(t)$ ensures that the successor state $s_t = o(a_t, o(\langle a_1, \ldots, a_{t-1}\rangle, s_0))$ is defined and the choice rule in Line 6 of Listing 2, together with the integrity constraints in $S(t)$ guarantee that

$$\{\langle X, V\rangle \mid \texttt{holds(X,V,t)} \in M\} = \{\langle X, V\rangle \mid X \in \mathcal{F}, V = o(\langle a_1, \ldots, a_t\rangle, s_0)(X)\}.$$

Thus (2) holds for each $1 \leq t \leq n$. Given `query(n)`, the integrity constraint in $Q(n)$ yields that $o(\langle a_1, \ldots, a_n\rangle, s_0) = s_*(X)$ for each $X \in \text{dom}(s_*)$ and therefore $\langle a_1, \ldots a_n\rangle$ is a sequential plan.

□

Theorem 2 (Correctness of parallel plans (Dimopoulos, Gebser, et al. 2019))**.** Let $\mathcal{A} = \langle A_1, \ldots, A_m \rangle$ be a sequence of sets of actions, $A_t \subseteq \mathcal{O}$ for $1 \leq t \leq m$. Then,

$\mathcal{A}$ is a $\forall$-/$\exists$-/R-step plan $\iff$ $P^p(m)$ has a stable model M such that
$\{\langle a, t\rangle \mid \texttt{occurs}(a, t) \in M\} = \{\langle a, t\rangle \mid a \in A_t, 1 \leq t \leq m\}$

Proof. We prove both directions separately.

($\Rightarrow$): Assume $\mathcal{A} = \langle A_1, \ldots, A_m \rangle$ is a $\forall$-/$\exists$-/*relaxed* $\exists$-step plan. For each $1 \leq t \leq m$, let s_t be the state defined by applying the set A_t to s_{t-1}. Due to the confluence of A_t, s_t is well-defined as:

$$s_t(X) = \begin{cases} a^e(X) & \text{for each } a \in A_t \text{ and } X \in \mathrm{dom}(a^e) \\ s_{t-1}(X) & \text{otherwise (inertia)} \end{cases}$$

Analogous to the proof of Theorem 1, consider any stable model M of $P^p(m)$ such that the action occurrences match the plan, i.e. $\{\langle a, t\rangle \mid \texttt{occurs}(a, t) \in M\} = \{\langle a, t\rangle \mid a \in A_t, 1 \leq t \leq m\}$. The fact rule in B, together with the choice rule and integrity constraints in $S(t)$, dictates the values of all fluents based on these actions. Thus, the set of `holds`-atoms in M is determined as:

$$\{\langle X, V, t\rangle \mid \texttt{holds}(X, V, t) \in M\} = \{\langle X, V, t\rangle \mid X \in \mathcal{F}, V = s_t(X), 0 \leq t \leq m\}.$$

We now distinguish the different cases of parallel plans:

Case 1: $\mathcal{A}$ is a $\forall$-step plan.
For $1 \leq t \leq m$, the atom $\texttt{single}(X, t)$ defined in $S^\forall(t)$ (Line 2 of Listing 5) is derived iff there exists some action $a \in A_t$ that invalidates its own precondition, i.e., $X \in \mathrm{dom}(a^c) \cap \mathrm{dom}(a^e)$ and $a^c(X) \neq a^e(X)$. Hence, $P^\forall(m)$ cannot have any stable model M apart from

$$\begin{aligned} M = I \cup &\{\texttt{occurs}(a, t) \mid a \in A_t, 1 \leq t \leq m\} \\ \cup &\{\texttt{holds}(X, V, t) \mid X \in \mathcal{F}, V = s_t(X), 0 \leq t \leq m\} \\ \cup &\{\texttt{single}(X, t) \mid a \in A_t, X \in \mathrm{dom}(a^c) \cap \mathrm{dom}(a^e), a^c(X) \neq a^e(X), 1 \leq t \leq m\} \\ \cup &\{\texttt{query}(m).\} \end{aligned}$$

such that $\{\langle a, t\rangle \mid \texttt{occurs}(a, t) \in M\} = \{\langle a, t\rangle \mid a \in A_t, 1 \leq t \leq m\}$.
It remains to show that M satisfies the integrity constraints in $S^\forall(t)$. Since for any

$1 \leq t \leq m$, the successor state $s_t = o(\langle a_1, \ldots, a_k \rangle, s_{t-1})$ is defined for *any* sequence $\langle a_1, \ldots, a_k \rangle$ with $\{a_1, \ldots, a_k\} = A_t$, no action can interfere with the precondition of another action. Consequently, if $\texttt{single}(X, t)$ is derived (meaning an action invalidates its own precondition on X), no other action can modify X, as this would create a conflict in at least one permutation. Therefore, the integrity constraint restricting the count of actions affecting X alongside $\texttt{single}(X, t)$ is satisfied. Analogously, the constraint requiring preconditions to remain stable (Line 1 of Listing 5) is satisfied since $s_{t-1}(X) = a^c(X)$ for each $a \in A_t$, $X \in dom(a^c)$ and $1 \leq t \leq m$. One can check that the remaining rules and integrity constraints in $S(t)$ are satisfied as well, so M is a stable model of $P^{\forall}(m)$.

Case 2: $\mathcal{A}$ is a $\exists$-step plan.
We analyze the encodings $P^{\exists}$ and P^{E} separately, starting with the acyclicity-based encoding P^{E}.
Analysis of P^{E}:
By definition of an $\exists$-step plan, for every timestep $1 \leq t \leq m$, there is a sequence of actions $\langle a_1, \ldots a_k \rangle$ with $\{a_1, \ldots, a_k\} = A_t$. A valid serialization requires that no action invalidates the precondition of a subsequent action. Formally, let $G_t = (A_t, E_t)$ be the conflict graph for timestep t, where an edge represents a destroyed precondition:

$$E_t = \{(a, a') \in A_t \times A_t \mid a \neq a', \exists X \in \mathrm{dom}(a^e) \cap \mathrm{dom}(a'^c) : a^e(X) \neq a'^c(X)\}.$$

Since a valid serialization exists, $\langle a_1, \ldots, a_k \rangle$ constitutes a topological ordering of G_t, i.e. there exists a sequence of applied actions. A directed graph possesses a topological ordering iff it is a Directed Acyclic Graph (DAG). Thus, G_t must be acyclic.
Now consider the candidate model M constructed as in Case 1:

$$\begin{aligned} M = I \cup \{&\texttt{occurs}(a, t) \mid a \in A_t, 1 \leq t \leq m\} \\ &\cup \{\texttt{holds}(X, V, t) \mid X \in \mathcal{F}, V = s_t(X), 0 \leq t \leq m\} \cup \{\texttt{query}(m)\}. \end{aligned}$$

The `#edge` directive in Listing 8 (Lines 2-3) generates an edge between two occurring actions (a, t) and (a', t) exactly if the condition for E_t is met. Consequently, the set of edges declared in the ASP model corresponds exactly to the union of all conflict graphs $\bigcup_{t=1}^{m} E_t$. Since each G_t is acyclic and the graphs for different timesteps $1 \leq t < t' \leq m$ are vertex-disjoint ($(a, t), \neq (a, t')$), the overall graph induced by M is acyclic. Therefore, the acyclicity constraint is satisfied, and M is a stable model of $P^{E}(m)$.
Analysis of $P^{\exists}$:
The rules defining `apply` and `ready` in $S^{\exists}(t)$ (Lines 2-5 in Listing 7) effectively traverse the dependency graph G_t. Since G_t is acyclic, there exists a topological ordering of the actions. The derivation of atoms proceeds along this ordering: Actions with no dependencies

(sinks in G_t) are derived as `apply` first, making them `ready`. This, in turn, satisfies the conditions for their predecessors. Because G_t is finite and acyclic, this inductive process reaches a fixpoint where $\texttt{apply}(a,t)$ and $\texttt{ready}(a,t)$ hold for all $a \in \mathcal{O}$ and $1 \leq t \leq m$. Consequently, the integrity constraint requiring all actions to be ready (Line 6 in Listing 7) is satisfied. Thus, $P^{\exists}(m)$ extends the base model M to:

$$M' = M \cup \{\texttt{apply}(a,t) \mid a \in \mathcal{O}, 1 \leq t \leq m\} \cup \{\texttt{ready}(a,t) \mid a \in \mathcal{O}, 1 \leq t \leq m\}$$

such that $\{\langle a,t\rangle \mid \texttt{occurs}(a,t) \in M'\} = \{\langle a,t\rangle \mid a \in A_t, 1 \leq t \leq m\}$. As established previously, the state transition rules are satisfied by M (and thus by M'). Since the additional constraints in $P^{\exists}$ regarding circularity are also satisfied by the completeness of the `ready` derivation, M' is indeed a stable model of $P^{\exists}(m)$.

Case 3: $\mathcal{A}$ is a *relaxed* $\exists$-step plan.
For each $1 \leq t \leq m$, by definition of a relaxed $\exists$-step plan, there exists a serialization $\langle a_1, \ldots, a_k\rangle$ of A_t such that the sequential execution is valid. This means $o(a_i, o(\langle a_1, \ldots, a_{i-1}\rangle, s_{t-1}))$ is defined for all $1 \leq i \leq k$.
Consequently, the preconditions of any action a_i are satisfied by the state resulting from the application of its predecessors in $\langle a_1, \ldots, a_n\rangle$. The rules in $S^R(t)$ (Listing 10) simulate this reachability:

(i) The atom $\texttt{reach}{>}(X,V,t)$ is derived for all values V that are either present in s_{t-1} or produced by an applied action.

(ii) The atom $\texttt{apply}(a,t)$ is derived for any action $a \in \mathcal{O}$ whose preconditions are satisfied by the reachable values.

(iii) The atom $\texttt{ready}(a,t)$ is derived for all $a \in \mathcal{O}$ (either because they are applied, or because they do not occur).

Since the serialization $\langle a_1, \ldots, a_k\rangle$ is valid, the least fixpoint of these rules guarantees that $\texttt{apply}(a,t)$ is derived for all $a \in A_t$.
Thus, the stable model M of $P^R(m)$ is:

$$\begin{aligned} M = I &\cup \{\texttt{occurs}(a,t) \mid a \in A_t, 1 \leq t \leq m\} \\ &\cup \{\texttt{holds}(X,V,t) \mid X \in \mathcal{F}, V = s_t(X), 0 \leq t \leq m\} \\ &\cup \{\texttt{reach}(X,V,t) \mid X \in \mathcal{F}, V \in \{s_{t-1}, s_t\}, 1 \leq t \leq m\} \\ &\cup \{\texttt{apply}(a,t) \mid a \in \mathcal{O} : a^c(X) \in \{s_{t-1}(X), s_t(X)\} \text{ for each } X \in \mathrm{dom}(a^c)\} \\ &\cup \{\texttt{ready}(a,t) \mid a \in \mathcal{O}, 1 \leq t \leq m\} \\ &\cup \{\texttt{query}(m)\} \end{aligned}$$

Since M contains $ready(a,t)$ for all actions, the integrity constraint in Line 7 of Listing 10 is satisfied. Furthermore, all state transition constraints are met by the definition of s_t.

Therefore, M is a stable model of $P^R(m)$.

($\Leftarrow$): Assume that M is a stable model of $P^p(m)$ such that $\{\langle a,t\rangle \mid \texttt{occurs}(a,t) \in M\} = \{\langle a,t\rangle \mid a \in A_t, 1 \leq t \leq m\}$.

For any $1 \leq t \leq m$, the choice rule in Line 6 of Listing 2 ensures that the set of `holds`-atoms induces a state function s_t defined by $s_t(X) = V \Leftrightarrow \texttt{holds}(X,V,t) \in M$.

Moreover, the rules and integrity constraints in $S(t)$ dictate the state transition. Specifically, the integrity constraint in Line 8 of Listing 2 ensure that for each $a \in A_t$ and $X \in \mathrm{dom}(a^e)$, $s_t(X) = a^e(X)$ holds. Since M is a stable model, contradicting effects on the same variable are impossible (as they would violate the constraints), which implies that A_t is confluent. For unaffected fluents, the integrity constraint in Line 9 of Listing 2 yield $s_t(X) = s_{t-1}(X)$ for each $X \in \mathcal{F} \setminus \bigcup_{a \in A_t} \mathrm{dom}(a^e)$.

Finally, given the fact `query`(m), the integrity constraint in $Q(m)$ ensures that $s_m(X) = s_*(X)$ for all $X \in \mathrm{dom}(s_*)$, satisfying the goal condition.

We now distinguish the different cases to show that A_t is $\forall$-/$\exists$-/*relaxed* $\exists$-step serializable in s_{t-1} based on the specific constraints of each encoding.

Case 1: To show: A_t is $\forall$-step serializable.

Let $\langle a_1, \ldots, a_k\rangle$ be an arbitrary sequence of actions such that $\{a_1, \ldots, a_k\} = A_t$. We prove by induction that for all $1 \leq i \leq k$, the transition $o(\langle a_1, \ldots, a_i\rangle, s_{t-1})$ is defined.

Base case: $i = 1$:

The integrity constraint checking preconditions (Line 1 of Listing 5) guarantees that a_1 is applicable in s_{t-1}, i.e., $o(a_1, s_{t-1})$ is defined.

Inductive step $i \mapsto i+1$:

Assume $o(\langle a_1, \ldots, a_i\rangle, s_{t-1})$ is defined. We must show that a_{i+1} is applicable in the resulting intermediate state $s_{temp} = o(\langle a_1, \ldots, a_i\rangle, s_{t-1})$. Consider any fluent X in the precondition of a_{i+1}. If $X \notin \bigcup_{1 \leq j \leq i} \mathrm{dom}(a_j^e)$, then no preceding action has modified X. Thus $s_{temp}(X) = s_{t-1}(X)$, and the precondition holds trivially because a_{i+1} is applicable in s_{t-1}. Therefore, we focus on the critical case where $X \in \mathrm{dom}(a_{i+1}^c) \cap \bigcup_{1 \leq j \leq i} \mathrm{dom}(a_j^e)$. This implies there exists a preceding action a_j that has a postcondition on X. We distinguish two cases based on the effect of a_{i+1} itself:

(i) $X \notin \mathrm{dom}(a_{i+1}^e)$:
The integrity constraint in Line 2 of Listing 5 requires that if a_{i+1} has a precondition on X but no postcondition, the value of X cannot change in the final state s_t. Since a_j writes X, we have $s_t(X) = a_j^e(X)$. The constraint implies $s_{t-1}(X) = s_t(X)$, meaning a_j effectively did not change the value relative to s_{t-1}. Thus $s_{temp}(X) = a_{i+1}^c(X)$.

(ii) $X \in \mathrm{dom}(a_{i+1}^e)$:
The integrity constraint in Line 4 of Listing 5 ensures that if an action (a_{i+1}) has a precondition on X and sets a conflicting postcondition $(a^c(X) \neq a^e(X))$, it is

marked as `single`. However, since another action a_j also acts on X (as established by the intersection), the integrity constraint in Line 5 would be violated. Since M is a stable model, this implies that $\texttt{single}(X,t) \notin M$ and $s_t(X) = a^e_{i+1}(X) = a^c_{i+1}(X) = s_{t-1}(X)$ must be the case.

In both cases, $s_{temp}(X) = a^c_{i+1}(X)$, so $o(\langle a_1, \ldots, a_{i+1}\rangle, s_{t-1})$ is defined. Therefore, by induction, the entire sequence is valid, and A_t is $\forall$-step serializable.

Case 2: To show: A_t is $\exists$-step serializable.
The integrity constraint in Line 1 of Listing 7 guarantees that every action $a \in A_t$ is applicable in s_{t-1}, i.e., $o(a, s_{t-1})$ is defined. Furthermore, the integrity constraint in Line 6 ensures that $\texttt{ready}(a,t) \in M$ for each $a \in \mathcal{O}$ and the rules in Lines 4-5 that $\texttt{apply}(a,t) \in M$ for all $a \in A_t$. The rule defining `apply` (Lines 2-3) imposes a dependency: an action a can only be applied if all actions a' whose preconditions it invalidates are already `ready`. Since M is a stable model, there exists a derivation sequence of the `apply` atoms that respects these dependencies. Let $\langle a_1, \ldots, a_k\rangle$ be the sequence of actions in A_t ordered by their derivation step. Consider any action a_i in the sequence and a fluent X in the precondition. Suppose that a preceding action a_j $(j < i)$ invalidates this precondition, i.e., $a^e_j(X) \neq a^c_i(X)$. By the definition of `apply` in the Lines 2-3, a_j would have to wait for a_i to become `ready` before it could be applied. This would imply that a_i is derived before a_j, contradicting $j < i$. Therefore, for all $j < i$, a_j does not invalidate a_i. Thus, $s_{t-1}(X)$ is either preserved $(s_{t-1}(X) = s_t(X))$ or modified confluently $(a^e_j(X) = a^c_i(X))$, ensuring $o(\langle a_1, \ldots, a_k\rangle, s_{t-1})$ is defined.

Regarding the alternative encoding using the `#edge` directive (Listing 8), the graph logic ensures serializability explicitly. For any $1 \leq t \leq m$, the directive defines the conflict graph $G_t = (A_t, E_t)$ where a directed edge $(a, a') \in E_t$ exists iff action a invalidates a precondition of action a'. Since $P^E(m)$ enforces that G_t is acyclic, there exists a sequence $\langle a_1, \ldots, a_k\rangle$ comprising the actions in A_t such that no edge connects any predecessor to a successor. Formally, for any a_i in the sequence, there is no a_j with $j < i$ such that $(a_j, a_i) \in E_t$. This absence of destructive edges from the past implies that for any X in the precondition, either no predecessor changes X or it is modified consistently, i.e. $s_{t-1}(X) = a^c_i(X)$ or $a^e_j(X) = a^c_i(X)$. Thus, $o(\langle a_1, \ldots, a_i\rangle, s_{t-1})$ is defined for all i, and in particular $o(\langle a_1, \ldots, a_k\rangle, s_{t-1})$ is defined, verifying that A_t is $\exists$-step serializable.

Case 3: To show: A_t is *relaxed* $\exists$-step serializable.
In view of the rule in Line 1 of Listing 10, a fluent value is reachable if $s_{t-1}(X) = V$, i.e., $\mathit{reach}(X, V, t) \in M$. The recursive structure of the rules for `apply` (Lines 3-4) and `reach` (Line 2) imposes a dependency ordering: An action a is only derived as `apply`(a,t) if all

its preconditions are already in the extension of the `reach` predicate.

Since M is a stable model, there exists a finite derivation sequence for all atoms in M. Let $\langle a_1, \ldots, a_k \rangle$ be the sequence of actions in A_t ordered by the step in which their corresponding `apply` atom was derived (or satisfied). We prove that this sequence is a valid serialization.

Consider any action a_i in this sequence and any fluent X in the precondition. The derivation of `apply`(a_i,t) requires that `reach`$(X, a_i^c(X), t)$ was previously derived. By the definition of the `reach` rules, this value must originate either from:

(i) The state s_{t-1} (Line 1), which implies $s_{t-1}(X) = a_i^c(X)$, or

(ii) The effect of a previously applied action a_j (Line 2), which implies $a_j \in \{a_1, \ldots, a_{i-1}\}$ and $a_j^e(X) = a_i^c(X)$.

In both cases, the precondition of a_i is satisfied by the state constructed from the initial state s_{t-1} and the effects of the predecessors $\langle a_1, \ldots, a_{i-1} \rangle$. Thus, $o(\langle a_1, \ldots, a_k \rangle, s_{t-1})$ is defined.

□

3 Method

3.1 Pipeline Overview

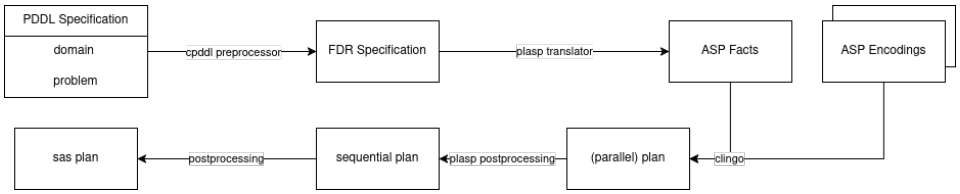


Figure 3.1: Pipeline for testing the different planning encodings

To systematically evaluate and compare the performance of the proposed parallel planning encodings against each other and the sequential baseline, we implemented an automated pipeline using the `lab` framework (Seipp et al. 2017). The benchmark[1] set consists of the unofficial collection of IPC instances provided by the Fast Downward project.

As illustrated in Figure 3.1, the workflow proceeds in four main stages:

1. **Preprocessing:** Starting with a planning task defined in PDDL (Planning Domain Definition Language), we use `cpddl` (Fišer 2025) to convert the problem into a Finite Domain Representation (FDR).
2. **Translation:** The `plasp` tool translates this FDR specification into a set of ASP facts. A detailed description of this translation process can be found in Dimopoulos, Gebser, et al. (2019).
3. **Solving:** In the core step, *clingo* combines these facts with our modified incremental encodings (sequential, ∀-step, ∃-step, or *relaxed* ∃-step). The solver then searches for stable models corresponding to valid plans.
4. **Validation:** Finally, to ensure correctness, the resulting ASP-based plan (represented by the `occurs` atoms) is converted back into a sequential plan format. We employ the post-processing encoding provided by *plasp* for serialization and subsequently format the output to be readable by the plan validator VAL[2].

Each component of this pipeline involves specific configurations and optimizations, which will be detailed in the following sections.

[1] https://github.com/aibasel/downward-benchmarks
[2] https://github.com/KCL-Planning/VAL

3.2 Encoding Adaptation

To integrate the theoretical encodings from Dimopoulos, Gebser, et al. (2019) presented in Section 2.2 into the automated pipeline, small changes are required to bridge the gap between the `plasp` output format and the internal predicates used by our logic programs. The necessary adaptations are shown in Listing 12.

```
1  #include <incmode>.
2  #program base.
3  fluent(X) :- variable(X).
4  fluent(X,V) :- initialState(X,V), fluent(X).
5  value(X,V) :- contains(X, V).
6  prec(A, X, V) :- precondition(A, X, V).
7  post(A, X, V) :- postcondition(A, _, X, V).
8  ...
9  :- fluent(X), #count{V : holds(X,V,0)} > 1.
10 ...
11 #program step(t).
12 ...
13 #show.
14 #show occurs(A, t): occurs(A, t).
```

Listing 12: Modifications to the core encoding

Lines 3-6 map the predicates generated by *plasp* (e.g., `initialState`, `contains`) to the predicates used in our encodings. Line 7 handles the action effects. By using the wildcard `_` for the second argument, we discard the specific effect identifiers. This simplification is chosen to reduce the complexity for the solver: explicitly tracking every effect could inflate the ground program and force *clingo* to manage a significantly larger set of atoms, which could degrade performance without necessarily improving the quality of the resulting plans. Line 9 performs a sanity check to ensure that the initial state is consistent (each fluent has at most one value). Finally, Lines 13-14 act as an output filter, ensuring that only the relevant `occurs` atoms are displayed in the final answer sets.

3.2.1 Post-Processing and Serialization

The stable models computed by *clingo* are output as sets of atoms (e.g., `occurs(action("move","a","b"), 1)`). Standard plan validators like *VAL*, however, require plans in a specific text-based format (often referred to as SAS plan or IPC format) structured as `time: (action args)`. To bridge

this gap, we implemented a lightweight post-processing module in Python. Briefly, the script iterates through the output string to identify all `occurs` atoms using regular expressions. For each match, it extracts the time step t and the action arguments, stripping internal quotes and formatting delimiters. Finally, the extracted pairs are written to a file in a line-by-line format compatible with the validator. This conversion ensures that the parallel plans found by our encodings can be verified against the original PDDL domain and problem specification.

3.3 Optimizations

3.3.1 Mutexes

To further optimize the solving process, we utilize invariant constraints derived during the translation process. Specifically, we incorporate *mutex groups* (mutually exclusive sets of facts) provided by the *cpddl* preprocessor.

Definition 8 ***mutex and mutex groups.*** A set of facts $M \subseteq \mathcal{F}$ is called

(i) a *mutex* $:\Longleftrightarrow$ for all reachable states $s : M \not\subseteq s$.

(ii) a *mutex group* $:\Longleftrightarrow$ for all reachable states $s : |M \cap s| \leq 1$.

Intuitively, a mutex group represents a set of facts where no two facts can be true simultaneously. Enforcing these invariants as integrity constraints can significantly prune the search space by detecting invalid partial states early. We extend our encoding with the rules shown in Listing 13.

```
1  #program base.
2  ...
3  mutex(G,X) :- mutexGroup(G), contains(G,X,V), fluent(X,V).
4  mutex(G)   :- mutexGroup(G), #count{X : mutex(G,X)} > 1.
5  :- mutex(G), #count{X,V : holds(X,V,0), contains(G,X,V)} > 1.
6  ...
7  #program step(t).
8  ...
9  :- mutex(G), #count{X,V : holds(X,V,t), contains(G,X,V)} > 1.
10 ...
```

Listing 13: Expansion for mutexes

The mutex information is provided by *plasp* via the predicates `mutexGroup/1` and `contains/3`. In the base program, we first define which fluents belong to a mutex group (Line 3), where a

group must contain at least two fluents (Line 4). The integrity constraint in Line 5 ensures that no more than one fluent of each mutex group holds in the initial state and the integrity constraint in Line 9 ensures the same for each state in timestep t.

3.3.2 Guess and Check Optimization

A crucial optimization for the $\exists$-step encodings is the *Guess and Check* approach. The core idea is to decouple the search for action sets from the computationally expensive verification of acyclicity. Instead of enforcing serializability directly during the search, we first "guess" a candidate solution that satisfies all local constraints (preconditions, effects, confluence) but ignores potential cycles. In a second step, we "check" this candidate for circular invalidations.

Formally, we decompose the problem into a pair of logic programs $\langle G, C \rangle$, where G is the *generator* or *guesser* and C is the *checker*. A stable model M of G represents a valid plan if and only if the checker program, when grounded with the actions from M, is **unsatisfiable** (i.e., no cycle can be derived).

To implement the generator G, Dimopoulos, Gebser, et al. (2019) proposes using the core program ($S(t)$ from Listing 2) combined with the precondition checks (Line 1 of Listing 3, 5, or 7) and the problem instance facts. Consequently, a stable model M of G guarantees that:

1. Preconditions hold at $t-1$ for all chosen actions.
2. Postconditions are applied and confluent (via Line 8 of Listing 2).

The only property remaining to be verified is the absence of circular invalidations (cycles in the conflict graph), where actions invalidates each other's preconditions (e.g., $a^e(X) \neq a'^c(X)$ and $a'^e(X') \neq a'^c(X')$).

This verification is performed by the program C, shown in Listing 14. The inputs to C are the action occurrences (`occurs/2`) from the candidate model M, along with the problem instance facts.

```
1  #include <incmode>.
2  #program check(t).
3  :- query(t), not cycle(t).
4  #program step(t).
5  prec(A, X, V, t) :- prec(A, X, V), occurs(A, t).
6  post(A, X, V, t) :- post(A, X, V), occurs(A, t).
7  action(A, t) :- occurs(A, t).
8  apply(A1, t) :- action(A1, t),
9      ready(A2, t): post(A1, X, V1, t), prec(A2, X, V2, t), A1 != A2 , V1 != V2.
10 ready(A, t) :- action(A, t), not occurs (A, t).
11 ready (A, t) :- apply(A, t).
12 cycle(t) :- action (A, t), not ready (A, t).
```

Listing 14: Encoding for detecting circular invalidation (Checker)

The checker works by attempting to construct a valid serialization order using the predicate `ready`:

- Like in the encoding of Listing 7, Lines 7-11 try to derive `ready`(A, t) for all actions that can be safely applied.
- Line 12 derives the atom `cycle`(t) if there exists any occurring action A that never becomes `ready`. This indicates that A is part of a invalidation cycle.
- Crucially, the program is designed to be **satisfiable** only if a cycle exists (Line 3).

Thus, if $C \cup M$ is satisfiable, a circular invalidation exists at some step t, meaning no valid permutation of the actions A_t exists. This implies the candidate is not serializable under the $\exists$-step semantics and must be rejected. Conversely, if $C \cup M$ is unsatisfiable, no cycle was found, and the candidate M is a valid plan.

3.3.3 Heuristic

To further optimize the solving process, we employ domain-independent heuristics using the `#heuristic` directive of *clingo*. Following the approach described by Dimopoulos, Gebser, et al. (2019), we modify the solver's search strategy by assigning specific attributes to atoms:

- *level* ($\in \mathbb{Z}$, default 0): Determines the decision priority. The solver selects atoms with higher levels before those with lower levels.
- *sign* ($\in \mathbb{Z}$, default 0): Determines the preferred truth value. A positive value (> 0) biases the solver to assign *true*, a negative value (< 0) to assign *false*. If the value is 0, the solver falls back to its default *sign* heuristic (e.g., VSIDS).

Listing 15 implements a *backward propagating* heuristic. The core idea is to leverage the goal condition and bias the search towards establishing the goal fluents as early as possible in the plan.

```
1 #program step(t).
2 #heuristic holds(X, V, t-1):    holds(X, V, t). [2147483647-t, true]
3 #heuristic holds(X, V, t-1):    not holds(X, V, t). [2147483647-t, false]
```

Listing 15: Backward propagating heuristic

The priority level is defined as $l = 2^{31} - 1 - t$ (the largest integer supported by *clingo*), assigning the highest priority to the earliest time steps (since smaller t yields a larger l). The logic in Lines 2 and 3 effectively projects state information backwards from t to $t-1$:

- **Positive Bias** (Line 2): If *clingo* determines that $holds(X, V, t)$ is true, the heuristic activates and suggests setting $holds(X, V, t-1)$ to true as well.
- **Negative Bias** (Line 3): Conversely, if $holds(X, V, t)$ becomes false, the heuristic biases $holds(X, V, t-1)$ to be false.

Intuitively, this encourages the solver to satisfy the goal state s_* by assuming (unless forced otherwise by actions) that the desired fluents already held in the previous step $t-1$. This creates a "pull" effect from the goal backwards to the initial state s_0.

3.4 Experimental Setup

To evaluate the performance of the different planning encodings and the effectiveness of the proposed optimizations, we performed a systematic comparison across four distinct experimental configurations.

3.4.1 Configurations and Encodings

We tested the sequential encoding alongside the ∀-step, both ∃-step and *relaxed* ∃-step encodings. Additionally, we evaluated the *Guess and Check* strategy with a fallback mechanism: if the Guess and Check approach fails to find a valid serializable plan (or implies a cycle), the system automatically falls back to a safe encoding (e.g. ∀-step). In such cases, the reported runtime is the sum of the time spent on the failed check and the time required by the fallback strategy. If this cumulative time exceeds the time limit, the instance is considered unsolved.

To isolate the impact of our optimizations, we ran the complete benchmark set in four configurations, varying the usage of mutexes and heuristics:

1. **Baseline:** No extra mutex detection, default solver heuristics.

2. **Mutexes (h2):** *cpddl* enabled with the h^2 option to compute additional mutex groups.
3. **Heuristics:** Using the backward propagating heuristic (Listing 15).
4. **Combined:** Both h^2 mutexes and backward heuristics enabled.

3.4.2 Benchmark Selection and Validation Criteria

Validation Criteria: For a planning task to be considered reliably *solved*, finding a candidate solution via *clingo* is necessary but not sufficient. We enforce a strict validation protocol: A run counts as solved if and only if:

1. The solver produces an answer set within the 30-minute time limit.
2. The extracted plan is successfully validated by the external validator *VAL*.

Any run where *clingo* reports a solution that *VAL* subsequently rejects is treated as an error and classified as unsolved.

Benchmark Domains: We evaluated our approach on a suite of benchmarks from the International Planning Competitions (IPC). The test set comprises domains used in previous competitions (e.g., IPC 1998-2018), specifically selected in their STRIPS formulations to ensure compatibility with our encodings. The selection covers a wide spectrum of planning complexity:

- *Classical Domains:* Standard benchmarks such as *Blocks*, *Logistics*, *Gripper*, and *Driverlog*.
- *Constraint-Heavy Domains:* Complex puzzles like *Sokoban*, *Freecell*, and *Parking*.
- *Recent Competitions:* Modern domains including *Agricola* (IPC 2018), *Snake* (IPC 2018), and *Quantum Layout* (IPC 2023).

This diverse set allows us to analyze the performance of parallel encodings across different problem structures (e.g., transportation, manipulation, construction).

3.4.3 Computational Environment

The experiments were executed on the *bwUniCluster 3.0* at the Karlsruhe Institute of Technology (KIT). We utilized compute nodes equipped with Intel Xeon Gold 6248R CPUs (3.0 GHz, 24 cores). To ensure comparability, each run was restricted to a single CPU core. We imposed a strict time limit of 30 minutes (1800 seconds) and a memory limit of 32 GiB for the execution of *clingo* and *cpddl* each. Runs exceeding these limits were aborted and recorded as unsolved (time-out).

4 Results

4.1 Overall Performance

To get an overview of the performance of the different encodings, we first analyze the aggregated results across all benchmark domains. In this section, we focus exclusively on the *baseline* configurations to establish a fundamental comparison. We compare the Sequential encoding (S) against the $\forall$-step ($\forall$), the $\exists$-step ($\exists$), the graph-based $\exists$-step (E), the *relaxed* $\exists$-step (R), and the Guess-and-Check strategy ($G\&C$).

Figure 4.1 illustrates the coverage (total number of solved instances) for each domain. For better readability, the data is split into two charts. Domains where no instances were solved by any algorithm are omitted. Figure 4.2 displays the average runtime. To ensure a fair comparison, this metric is calculated only on the intersection of instances that were successfully solved by *all* algorithms. Consequently, domains where no common set of solved instances exists are excluded from the runtime analysis.

4.1.1 Coverage Analysis

The coverage results in Figure 4.1 reveal several trends regarding the effectiveness of parallel planning.

Sequential vs. Parallel Performance: In domains that allow for a high degree of concurrency, parallel encodings significantly outperform the sequential approach. Notable examples include *airport*, *woodworking*, and both *logistics* variants. For instance, in *woodworking*, the sequential encoding (S) solves only 7 out of 50 instances, whereas all parallel encodings (with the exception of Guess-and-Check) solve the entire set. Similarly, in *floortile*, the sequential encoding fails to solve any instance (0/40), while the *relaxed* $\exists$-step encoding achieves a coverage of 38/40. This confirms that the reduced time horizon T enabled by parallel semantics can effectively mitigate the search depth explosion inherent to sequential planning. However, in more complex or inherently sequential domains like *agricola* or *snake*, this performance advantage is not observed.

Comparison of Parallel Encodings: Comparing the different parallel semantics reveals a trade-off between *plan flexibility* (allowing shorter horizons) and *encoding complexity* (grounding and solving overhead):

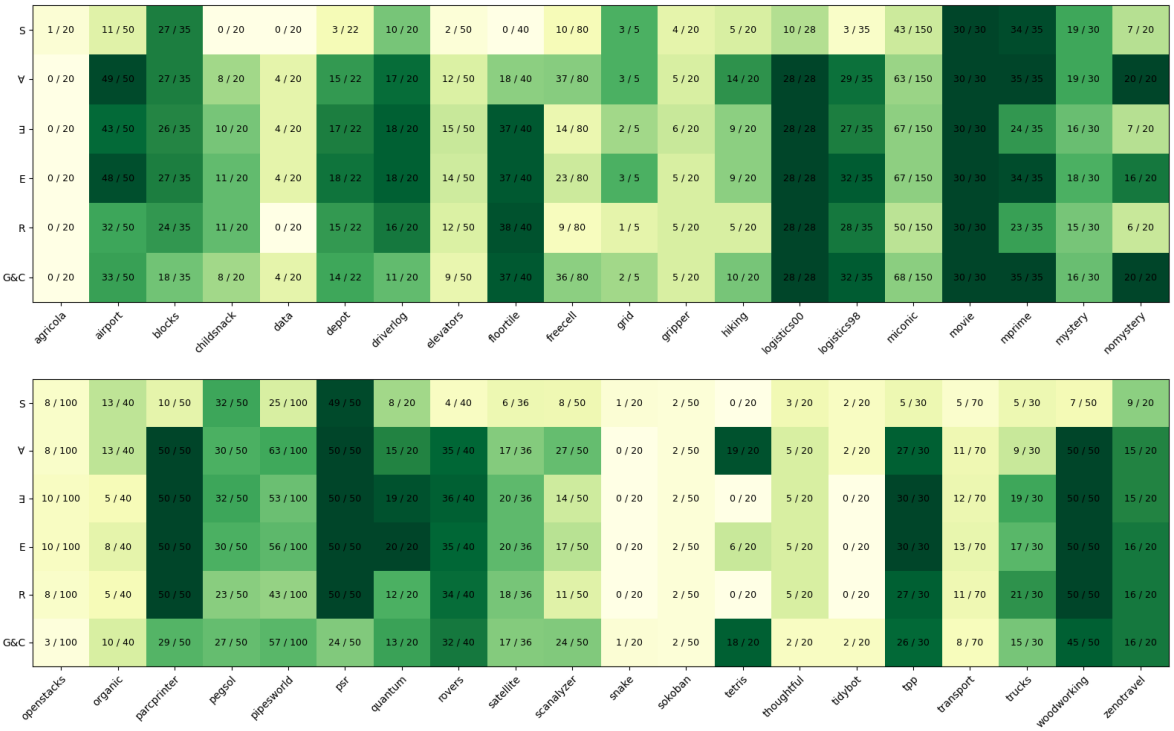


Figure 4.1: Coverage analysis: Number of solved instances per algorithm across benchmark domains. The heatmap indicates the absolute number of solved instances (left) out of the total available instances (right).

- **∀-step (∀):** Although it enforces the strictest requirements for parallel execution, the ∀-step encoding shows better performance in several domains (e.g., *tetris*, *parcprinter*), often outperforming even the more flexible variants in domains like *hiking* or *scanalyzer*. This suggests that for many IPC domains, the capability to interleave dependent actions is not critical. Consequently, the advantage of the ∀-step semantics lies in its compactness: it requires fewer constraints and auxiliary atoms than the ∃-step variants, thereby significantly reducing the burden on the grounder and solver (Dimopoulos, Gebser, et al. 2019).
- **∃-step (∃ /E):** The strength of the ∃-step semantics is seen in domains with complex dependencies. A prime example is *floortile*, where the strict ∀-step encoding solves only 18 out of 40 instances, whereas the ∃-step encoding achieves a coverage of 37/40. This demonstrates that the increased flexibility of the ∃-step semantics enables the discovery of plans in constrained environments where the conservative ∀-step approach fails. Regarding the implementation variants, the graph-based alternative (*E*) generally performs on par with or marginally better than the standard constructive encoding (∃), showing no significant deviation in total coverage.
- ***Relaxed* ∃-step (R):** The relaxed encoding typically yields lower coverage compared to the standard ∃-step variants, as observed in domains like *quantum-layout*. Although

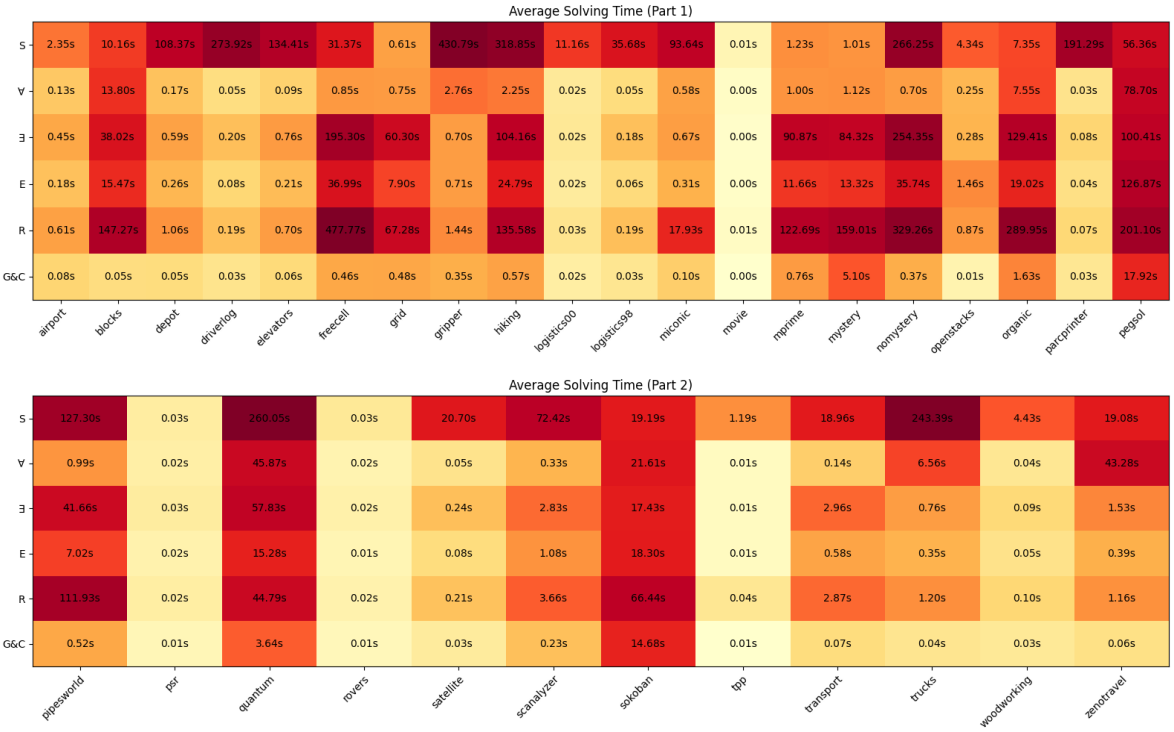


Figure 4.2: Runtime analysis: Average solving time (in seconds) for successfully solved instances. Red cells indicate higher runtimes, while light yellow cells indicate faster performance.

the *relaxed* ∃-step encoding theoretically enables the shortest plan horizons by allowing actions to support each other within the same step, the computational cost is significant. The recursive definition of the reachability predicates (`reach/3`) probably increases the complexity of the ground program. Consequently, the overhead of solving these dense constraints often outweighs the benefits gained from a reduced time horizon.

- **Guess-and-Check (*G&C*):** The performance of the Guess-and-Check strategy is generally inferior to the direct encodings (∃ and *E*). Although separating the cycle check from the plan generation reduces the complexity of the generator, the overhead of invoking the solver twice significantly impacts runtime. Crucially, when the validity check fails (i.e., the guessed plan contains a cycle), the pipeline must trigger the fallback mechanism. In these cases, the runtime accumulates (time for failed G&C + time for fallback), often leading to timeouts on instances that the pure ∀-step or ∃-step encodings could solve directly within the time limit. Consequently, this strategy proves less robust for general-purpose planning compared to the integrated approaches.

4.1.2 Runtime Analysis

Overview across Domains

Figure 4.2 provides an overview of the average runtimes across the evaluated domains. The sequential encoding (S) generally exhibits significantly higher runtimes compared to the parallel approaches. This discrepancy is particularly pronounced in domains with high concurrency potential, such as *Gripper*, where the average runtime for S is approximately 430 seconds, whereas the ∀-step encoding requires less than 3 seconds. This confirms that the reduced plan horizon in parallel planning typically translates to reduced solving time.

Among the parallel variants, the ∀-step encoding consistently achieves the lowest runtimes, rarely exceeding 100 seconds even on harder instances. This performance advantage can be attributed to the compactness of the encoding, which imposes the least burden on the grounder and solver.

The ∃-step encodings (∃ and E) display higher variability. While they allow for shorter plan lengths, the increased complexity of the compatibility checks often leads to longer solving times compared to ∀-step, particularly in dependency-heavy domains like *Freecell*.

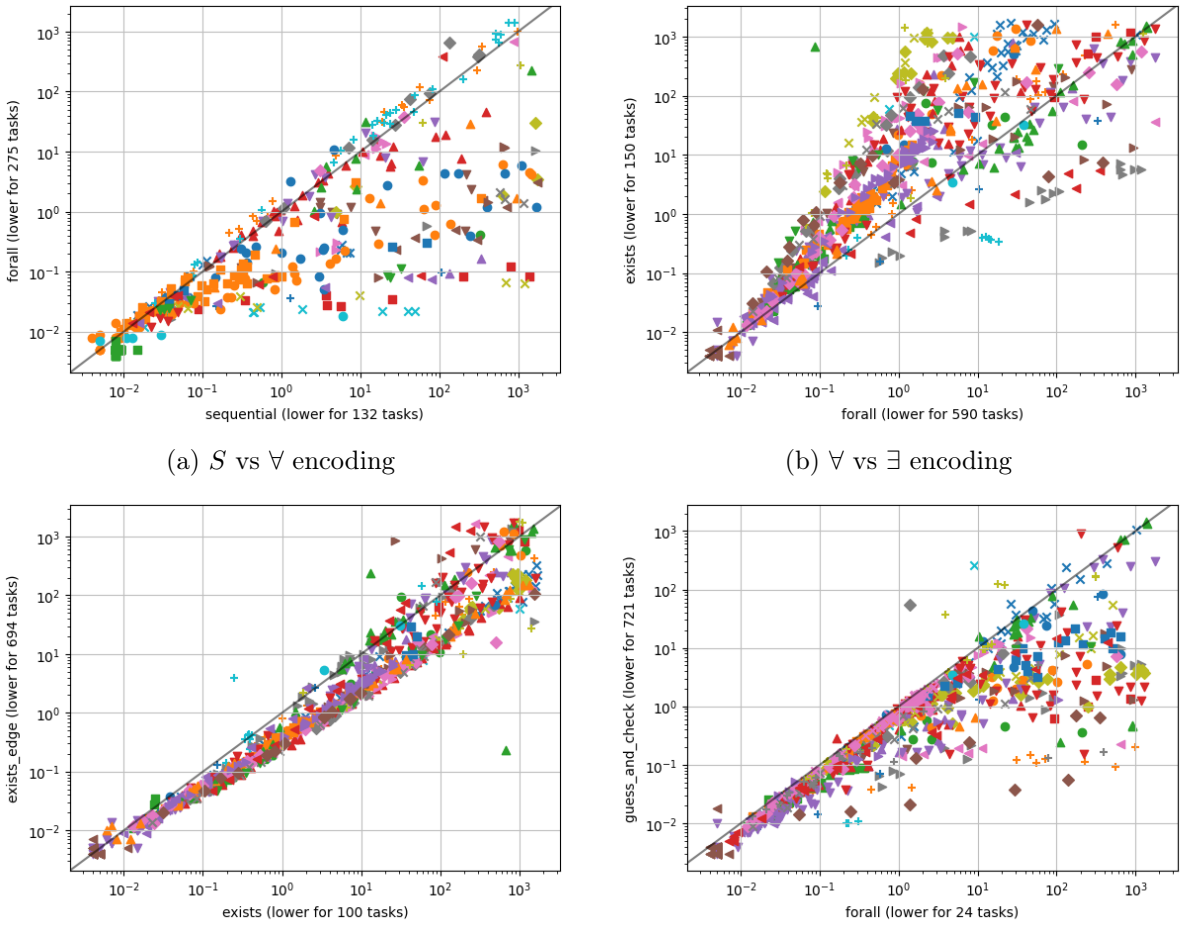
Detailed Pairwise Comparison

To investigate these performance characteristics and implementation differences in greater detail, Figure 4.3 provides pairwise scatter plots on common solved instances.

∃-step Implementation Variants (Fig. 4.3c): Comparing the two implementation variants reveals a distinct advantage for the graph-based approach. As illustrated in Figure 4.3c, the alternative ∃-step encoding (E) outperforms the standard constructive encoding (∃) in the majority of instances. Notably, in the *Nomystery* domain, E achieves a runtime reduction of over 200 seconds compared to ∃ on the largest instances, validating the efficiency of the `#edge` constraints.

∀-step vs. ∃-step (Fig. 4.3b): The direct comparison between the two main parallel semantics highlights the trade-off mentioned above. Although the ∃-step encoding is theoretically more flexible, the scatter plot shows a cluster of points above the diagonal. This confirms that the ∀-step encoding is frequently faster, suggesting that the overhead of grounding the complex ∃-step compatibility constraints outweighs the benefits of a slightly reduced horizon.

∀-step vs. Guess-and-Check (Fig. 4.3d): Finally, the comparison with the Guess-and-Check strategy ($G\&C$) challenges the assumption that the iterative approach necessarily incurs a high overhead. As shown in Figure 4.3d, the performance is largely on par with the ∀-step encoding, with $G\&C$ frequently achieving even faster runtimes (points situated below the diagonal). This efficiency can be attributed to the reduced complexity of the *generator* component. Since the



(a) S vs $\forall$ encoding

(b) $\forall$ vs $\exists$ encoding

(c) $\exists$ vs E encoding

(d) $\forall$ vs $G\&C$ encoding

Figure 4.3: Runtime comparison between different encodings. Each point represents an instance solved by both encodings, with the x-coordinate indicating the runtime of the first encoding and the y-coordinate that of the second. Points below the diagonal indicate better performance of the first encoding.

generator ignores circular dependency constraints, it produces a significantly smaller ground program than the full $\forall$-step encoding. In domains where circular conflicts are rare, the first "guessed" plan is often valid. Consequently, the combined runtime of the lightweight generator and the subsequent check is often lower than solving the stricter, more complex constraints of the $\forall$-step encoding directly.

4.2 Impact of Optimizations

To evaluate the effectiveness of the implemented optimizations, we compared four configurations: the baseline encoding, the encoding with h^2-mutexes, with the backward-propagating heuristic, and the combined configuration.

Heuristic The backward-propagating heuristic did not yield a significant improvement in coverage or runtime across the evaluated domains. This suggests that while the heuristic may guide the search towards goal-relevant states, it does not substantially reduce the search space in a way that translates to faster solving times.

Mutexes In almost all encodings, the inclusion of h^2-mutexes led to a reduction in runtime and an increase in coverage. This is paritcularly seen in the *agricola* domain, where beforehand only 1 instance was solved by the sequential encoding, now 7 instances are solved by the sequential encoding with the addition of mutexes and even 5 with the ∀-step encoding and the guess and check strategy. Regarding the runtime, the mutexes often reduce the solving time in domains and encoding with high solving times before. However, the trade-off is that cpddl takes significantly more time to preprocess the instance and compute the mutexes, which can lead to an overall increase in runtime for some instances. Overall, the use of mutexes impacts the coverage positivley and the effect of the runtime is slightly possitive as well.

4.3 Impact of Optimizations

To evaluate the effectiveness of the implemented optimizations, we compared the baseline encoding against configurations equipped with h^2-mutexes, the backward-propagating heuristic, and a combination of both. Figure 4.4 provides a detailed runtime comparison of these configurations.

Heuristic Guidance (Fig. 4.4a): Contrary to expectations, the backward-propagating heuristic did not yield a significant improvement in coverage or runtime. As illustrated in Figure 4.4a, the data points align almost perfectly with the diagonal, indicating negligible deviation from the baseline performance. This suggests that the search from the goal backwards do not substantially reduce the search time compared to *clingo*'s internal heuristic.

Impact of more Mutexes (Fig. 4.4c & 4.4d): In contrast, the inclusion of the h^2-algorithm for *cppdl* resulted in a measurable performance gain. The impact is particularly pronounced in highly constrained domains such as *Agricola*. In this domain, the sequential encoding solves 7 instances (compared to only 1 in the baseline) and the ∀-step encoding improves from 0 to 5 solved instances.

Analyzing the runtime reveals a distinct trade-off between solving power and preprocessing overhead:

- **Solver Efficiency (Fig. 4.4c):** When looking exclusively at the solver runtime (*clingo* time), the additional mutexes provide a drastic speedup. Most points lie well below the diagonal, showing that the mutex constraints effectively prune the search space.

- **Total Runtime (Fig. 4.4d):** When accounting for the full pipeline duration (including *cpddl* preprocessing), the picture changes. For many instances, the overall runtime does not significantly improve. In fact, for easier instances, the overhead of longer searching for mutexes outweighs the time saved during the plan search, leading to a longer total runtime compared to the baseline. This effect is particularly pronounced in the ∀-step encoding, which is already compact and thus benefits less from additional pruning. In comparison, for the *relaxed* ∃-step encoding, the total runtime with more mutexes is often lower than the baseline, indicating that "heavier" encodings profit more from the invariants.

Combined Configuration (Fig. 4.4b): As expected, adding the heuristic on top of the deeper mutex search does not yield further improvements. Figure 4.4b compares the encoding with the mutex optimization against the combined version. The almost perfect alignment with the diagonal confirms that the heuristic remains ineffective even in the presence of more mutexes.

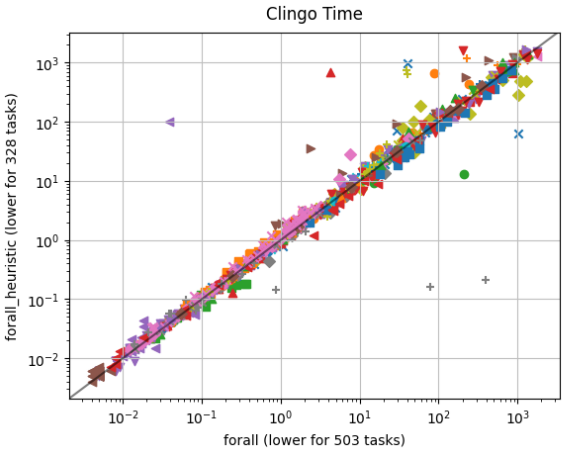
Figure 4.4b confirms that the system's performance is driven almost exclusively by the constraints generated by the h^2 algorithm.

4.4 Summary of Results

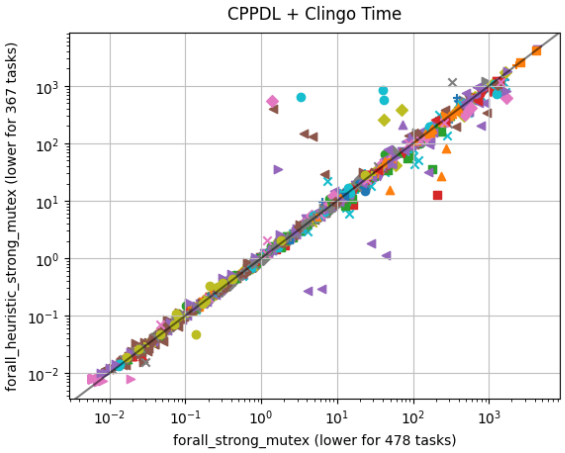
The experimental evaluation leads to three main conclusions regarding the performance of ASP-based planning encodings:

1. **Parallelism pays off:** Parallel encodings (∀-step and ∃-step) significantly outperform the sequential baseline in almost all standard benchmarks. The reduction of the planning horizon T is the primary driver for this efficiency.
2. **Simplicity beats flexibility:** Among the parallel variants, the strict ∀-step encoding proves to be the most robust. While the ∃-step semantics theoretically allow for shorter plans, the associated grounding overhead often negates this advantage. However, when ∃-step is required, the graph-based implementation (E) is superior to the constructive approach.
3. **Importance of Preprocessing:** The integration of invariants, specifically h^2-mutexes, is crucial for solving highly constrained instances. In contrast, simple domain-independent heuristics (like backward propagation) fail to provide a tangible benefit over the solver's built-in strategies.

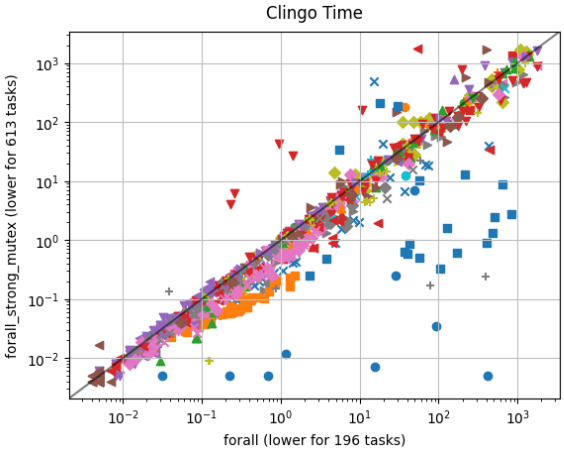
These findings suggest that future improvements should focus on compact encodings (like ∀-step) combined with rigorous preprocessing, rather than increasingly complex relaxed semantics.



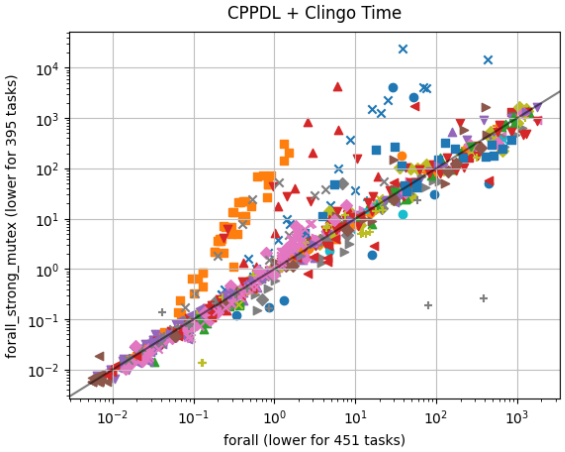
(a) Effect of Heuristic (Baseline vs. Heur.)



(b) Heuristic added to Mutexes



(c) Mutex Impact: Solver Time Only



(d) Mutex Impact: Total Time (incl. Prep.)

Figure 4.4: Runtime comparison of optimization techniques. Top row: The heuristic shows negligible impact (points on diagonal). Bottom row: Mutexes significantly reduce solver time (c), but preprocessing overhead affects total time on easy instances (d).

Bibliography

Gelfond, Michael and Vladimir Lifschitz (1988). "The Stable Model Semantics for Logic Programming". In: *Proceedings of International Logic Programming Conference and Symposium*. Ed. by Robert Kowalski, Bowen, and Kenneth. MIT Press, pp. 1070–1080. URL: http://www.cs.utexas.edu/users/ai-lab?gel88.

Dimopoulos, Yannis, Bernhard Nebel, and Jana Koehler (Oct. 1997). "Encoding Planning Problems in Nonmonotonic Logic Programs". In: ISBN: 978-3-540-63912-1. DOI: 10.1007/3-540-63912-8_84.

Helmert, M. (2006). "The Fast Downward Planning System". In: *Journal of Artifical Intelligence Research* 26, pp. 191–246.

Rintanen, Jussi, Keijo Heljanko, and Ilkka Niemelä (2006). "Planning as satisfiability: parallel plans and algorithms for plan search". In: *Artificial Intelligence* 170.12, pp. 1031–1080. ISSN: 0004-3702. DOI: 10.1016/j.artint.2006.08.002. URL: https://www.sciencedirect.com/science/article/pii/S0004370206000774.

Wehrle, Martin and Jussi Rintanen (Dec. 2007). "Planning as Satisfiability with Relaxed ∃-Step Plans." In: vol. 4830, pp. 244–253. ISBN: 978-3-540-76926-2. DOI: 10.1007/978-3-540-76928-6_26.

Gebser, Martin, Roland Kaminski, Benjamin Kaufmann, and Torsten Schaub (May 2014). *Clingo = ASP + control: Preliminary report*. Tech. rep. 1405.3694. arXiv. URL: https://arxiv.org/abs/1405.3694v1.

Bomanson, Jori et al. (Nov. 2016). "Answer Set Programming Modulo Acyclicity*". In: *Fundamenta Informaticae* 147, pp. 63–91. DOI: 10.3233/FI-2016-1398.

Gebser, Martin, Roland Kaminski, Benjamin Kaufmann, Max Ostrowski, et al. (2016). "Theory Solving Made Easy with Clingo 5". In: *Technical Communications of the 32nd International Conference on Logic Programming (ICLP 2016)*. Ed. by Manuel Carro et al. Vol. 52. Open Access Series in Informatics (OASIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2:1–2:15. ISBN: 978-3-95977-007-1. DOI: 10.4230/OASIcs.ICLP.2016.2. URL: https://drops.dagstuhl.de/entities/document/10.4230/OASIcs.ICLP.2016.2.

Seipp, Jendrik et al. (2017). *Downward Lab*. https://doi.org/10.5281/zenodo.790461.

Dimopoulos, Yannis, Martin Gebser, et al. (Jan. 2019). "plasp 3: Towards Effective ASP Planning". In: *Theory and Practice of Logic Programming* 19.3, pp. 477–504. ISSN: 1475-3081. DOI: 10.1017/s1471068418000583. URL: http://dx.doi.org/10.1017/S1471068418000583.

Fišer, Daniel (Nov. 2025). *cpddl*. Version 1.5. DOI: 10.5281/zenodo.16423302. URL: https://doi.org/10.5281/zenodo.16423302.